

React

Query



Introduction

React Query is a library that makes data fetching, caching, and updating server state in your React apps much easier. Instead of manually managing data with hooks like `useState` and `useEffect`, React Query takes care of the data lifecycle, allowing you to focus on building your UI.

Installation

Install React Query in your project with:

```
terminal
```

```
npm install @tanstack/react-query
```

Explanation:

This command downloads React Query from npm. Once installed, you can import its hooks and providers to start managing remote data seamlessly.



Setting Up React Query

Before using any hooks, you need to set up a context for React Query by wrapping your app with **QueryClientProvider**:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import App from './App';

const queryClient = new QueryClient();

function Root() {
  return (
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  );
}

export default Root;
```

Explanation:

- **QueryClient**: This is the central manager for all your queries. It handles caching, background updates, and more.
- **QueryClientProvider**: This provider makes the query client available to your entire app. Any component inside this provider can use React Query hooks like `useQuery` and `useMutation`.



Fetching Data with **useQuery**

Use useQuery to fetch and cache data from an API:

```
Users.jsx

import { useQuery } from '@tanstack/react-query';
import axios from 'axios';

const fetchUsers = async () => {
  const { data } = await axios.get('https://jsonplaceholder.typicode.com/users');
  return data;
};

function Users() {
  const { data, isLoading, error } = useQuery({ queryKey: ['users'], queryFn: fetchUsers });

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```



Explanation:

- useQuery Parameters:
 - **queryKey**: A unique key (['users'] in this case) that identifies the data. It helps React Query manage caching and refetching.
 - **queryFn**: The function that fetches the data (here, fetchUsers uses Axios to get data from a placeholder API).
- Query States:
 - **isLoading**: True while the data is being fetched.
 - **error**: Contains error details if the request fails.
 - **data**: Contains the fetched data once the request is successful.
- UI Logic:
 - While loading, it displays "Loading...".
 - On error, it displays an error message.
 - Once the data is available, it renders a list of users.



Mutations with **useMutation**

Use useMutation to perform operations like adding, updating, or deleting data:

```
import { useMutation, useQueryClient } from '@tanstack/react-query';
import axios from 'axios';

const addUser = async (user) => {
  const { data } = await axios.post('https://jsonplaceholder.typicode.com/users', user);
  return data;
};

function AddUser() {
  const queryClient = useQueryClient();
  const mutation = useMutation({
    mutationFn: addUser,
    onSuccess: () => {
      // Invalidate and refetch queries after mutation is successful
      queryClient.invalidateQueries(['users']);
    },
  });

  return (
    <button onClick={() => mutation.mutate({ name: 'New User' })}>
      Add User
    </button>
  );
}
```



Explanation:

- useMutation Parameters:
 - **mutationFn**: A function that sends data to the server (here, `addUser` posts a new user).
 - **onSuccess**: A callback that runs after the mutation completes successfully. In this example, it calls `queryClient.invalidateQueries(['users'])` to refresh the users list.
- **queryClient.invalidateQueries**: This tells React Query that the data associated with the key `['users']` is now outdated, prompting a refetch.
- UI Logic:
 - When the button is clicked, `mutation.mutate` triggers the API call to add a user.
 - After adding, the list of users updates automatically.



Common Query States

React Query provides several states to help manage the lifecycle of your data:

- **isLoading**: Indicates that the data is currently being fetched.
- **error**: Contains error details if fetching fails.
- **data**: Holds the successfully fetched data.
- **isFetching**: True when background refetching is occurring, even if data is already available.

Explanation:

These states allow you to build a robust UI by showing loaders, handling errors gracefully, and updating the display as new data becomes available.





Hopefully You Found It Usefull!

“Be sure to save this post so you
can come back to it later

like

Comment

Share